IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Introducción a Python

De la hoja de cálculo al programa: lo justo para empezar el reto

Sprint 4 · S04_01 · Para empezar el sprint y para repasar en casa

POR QUÉ PYTHON

De la hoja de cálculo al programa

Hoja de cálculo

Se hace **a mano**.

Se rompe sin avisar.

Nadie sabe quién tocó qué.

No se puede repetir igual.

Programa en Python

Se **ejecuta** con un clic.

Falla con un mensaje claro.

Queda guardado y con historial.

Cualquiera lo repite y lo comprueba.

Caso del sprint: TurisData calcula sus cifras a mano; vamos a convertir esos cálculos en kpis.py.

DÓNDE SE EJECUTA PYTHON

Tres sitios, una regla de oro

Dónde	Cuándo	Ojo con
Google Colab	En el aula: no se instala nada	Al cerrar la sesión **se pierde el estado**
Jupyter o VS Code	En tu equipo, con tus ficheros	Hay que instalar Python y las librerías
Terminal: `python kpis.py`	Para el script del reto	Es lo que usará TurisData

Un **cuaderno** mezcla texto y celdas de código. Las celdas recuerdan lo que ejecutaste **en el orden en que lo hiciste**, no en el orden en que aparecen.

Algo raro: reiniciar el entorno y ejecutar todo de arriba abajo.

VARIABLES Y TIPOS · S04_01

Una variable es una caja con nombre

```
habitaciones, precio, tasa = 3, 85.0, 2.0
total = habitaciones * (precio + tasa)
print(f"{habitaciones} hab.: {total:.2f} EUR")
# 3 hab.: 261.00 EUR
```

Tipo	Qué es	Ejemplo
int	Entero	3
float	Decimales	85.0
str	Texto	"doble"
bool	Verdadero o falso	True

= **asigna** (mete un valor en la caja). **print** lo muestra. En el f-string, **{total:.2f}** es «con 2 decimales». Los decimales llevan **punto**, no coma.

Comparar no es asignar

Operador	Hace	Ejemplo
/	División normal	**7 / 2 → 3.5**
//	División entera	**7 // 2 → 3**
%	Resto	**7 % 2 → 1**
==	¿Son iguales?	**canal == "web"**
and or not	Combinar condiciones	**a and b**

```
bruto = "  Agencia Online "
canal = bruto.strip().lower().replace(" ", "_")
# "agencia_online"
```

Los datos reales vienen sucios: los métodos de texto se **encadenan**.

Error clásico: if canal = "web": da error. == compara, = asigna.

DECIDIR · S04_01

if, elif, else: gana el primer caso que se cumple

```
def tipo_reserva(dias):
    if dias < 7:
        return "inmediata"
    elif dias < 30:
        return "corta"
    return "anticipada"
```

dias	Resultado
3	inmediata
15	corta
30	anticipada
45	anticipada

Python lee **de arriba abajo** y se queda con el primer caso que se cumple. Los dos puntos **:** y la **sangría de 4 espacios** son obligatorios.

REPETIR · S04_01

for recorre; while repite mientras se cumpla algo

Una estancia de 4 noches: la primera cuesta 50 € y cada noche sube 10 €.

```
total, precio = 0, 50
for noche in range(4):
    total += precio + 10 * noche
# total = 260
```

Vuelta	noche	Se suma	total
1	0	50	50
2	1	60	110
3	2	70	180
4	3	80	**260**

range(4) da **0, 1, 2, 3**: empieza en 0 y **no incluye** el 4. **total += x** es **total = total + x**.

Dentro de un while algo tiene que cambiar, o no acaba nunca.

FUNCIONES · S04_02

Una función es una receta con nombre

```
def precio_final(noches, precio, descuento=0.0):
    """Importe de la estancia con descuento."""
    return round(noches * precio * (1 - descuento), 2)
assert precio_final(3, 90) == 270.0
assert precio_final(5, 90, 0.2) == 360.0
```

return

Devuelve el valor. **print** solo lo muestra.

assert

Silencio = bien. Si es falso, salta **AssertionError**.

descuento=0.0 es un parámetro **por defecto**: si no lo das, vale 0. El texto entre triples comillas es el **docstring**. Una función que solo hace **print** **no se puede reutilizar** en otros cálculos.

Lista, tupla, diccionario y conjunto

Estructura	Se escribe	Rasgo clave	Ejemplo
Lista	[ ]	Ordenada y **modificable**	[4, 2, 6, 3, 5]
Tupla	( )	Ordenada e **inmutable**	(28.46, -16.25)
Diccionario	{ }	**Clave → valor**	{"canal": "web", "noches": 2}
Conjunto	{ }	**Sin repetidos**	{"web", "directo"}

```
lista = [4, 2, 6, 3, 5]
lista[0]            # 4 (la primera posición es la 0)
lista[-1]           # 5 (el último)
sum(lista[-2:])     # 8 (los dos últimos: 3 + 5)
```

Cada diccionario es una fila

```
reservas = [
    {"canal": "web",     "noches": 2, "precio": 80.0, "cancelada": 0},
    {"canal": "web",     "noches": 3, "precio": 90.0, "cancelada": 1},
    {"canal": "directo", "noches": 4, "precio": 75.0, "cancelada": 0},
]
ingresos = [r["noches"] * r["precio"] for r in reservas if r["cancelada"] == 0]
# [160.0, 300.0]   -> media 230.0
```

Fila y columna

Cada diccionario es **una fila**; cada clave, **una columna**, como en una hoja de cálculo.

Comprensión de lista

«Para cada reserva **r**, si no está cancelada, calcula noches × precio»: un bucle **en una línea**.

CONTAR CON UN DICCIONARIO · S04_03

Tasa de cancelación por canal

```
total, canceladas = {}, {}
for r in reservas:
    c = r["canal"]
    total[c] = total.get(c, 0) + 1
    canceladas[c] = canceladas.get(c, 0) + r["cancelada"]
tasa = {c: round(100 * canceladas[c] / total[c], 1) for c in total}
# {'web': 50.0, 'directo': 0.0}
```

get(c, 0)

«Dame el valor de **c** y, si todavía no existe, dame **0**».

El resultado

Del canal web hay 2 reservas y 1 cancelada: **50 %**. Directo: 1 y 0: **0 %**.

FICHEROS · S04_04

Leer un CSV y guardar un JSON

```
import csv, json
with open("reservas.csv", newline="", encoding="utf8") as f:
    filas = list(csv.DictReader(f))        # lista de diccionarios
noches = int(filas[0]["noches"])           # hay que CONVERTIR
with open("kpis.json", "w", encoding="utf8") as f:
    json.dump(kpis, f, ensure_ascii=False, indent=2)
```

Todo llega como texto

En el CSV pone **3**, pero Python lee **"3"**. **"3" + "4"** da **"34"**; con **int** da **7**.

with open

Cierra el fichero solo al terminar el bloque.

Tildes y legibilidad

encoding="utf8", **ensure_ascii=False** e **indent=2**.

CUANDO ALGO FALLA · S04_04

try/except: gestionar el error en vez de explotar

```
def a_float_seguro(texto, defecto=None):
    try:
        return float(texto)
    except (ValueError, TypeError):
        return defecto
```

Llamada	Resultado
a_float_seguro("12.5")	12.5
a_float_seguro("abc")	None
a_float_seguro(None, 0.0)	0.0

Un error concreto

Nunca un **except:** vacío: **esconde** también los fallos que no esperabas.

Contar lo descartado

Tirar filas malas **sin avisar** deja los KPIs mal y nadie lo sabe.

CUANDO SALE ROJO · S04_04

Lee la última línea del error

Mensaje	Qué significa	Qué mirar
NameError	Variable que no existe	¿La ejecutaste antes? ¿Mal escrita?
SyntaxError	Falta un **:**, comilla o paréntesis	La línea anterior a la señalada
IndentationError	Sangría mal	Usa siempre 4 espacios
TypeError	Mezclas tipos (**"3" + 4**)	Convierte con **int()** o **float()**
KeyError	Clave que no existe	El nombre exacto de la columna
FileNotFoundError	No encuentra el fichero	Carpeta y nombre del archivo
ZeroDivisionError	Divides entre 0	Que el total no sea 0

EL RETO · MISIONES 3 A 5

Pruebas primero, y un commit por cada paso

1 · En rojo

Ejecuta **python test_kpis.py**: **tiene que fallar**, todavía no hay código.

2 · Una función

Rellena **solo una** función de **kpis.py**.

3 · En verde

Vuelve a ejecutar: esa prueba **pasa a verde**.

4 · Commit

Una «foto» con un mensaje que diga **qué cambia y por qué**.

Función de **kpis.py**	Qué hace
leer_reservas(ruta)	Lee, valida y devuelve **(reservas, avisos)**
tasa_cancelacion_por_canal	% de canceladas por canal, con 1 decimal
ingreso_medio_por_reserva	Media de noches × precio de las **no canceladas**
exportar_json	Guarda el resultado con tildes y sangría

DÓNDE PRACTICAR CADA COSA

Cada idea tiene su misión y su cuaderno

Si no entiendes...	En clase	En casa
Variables, tipos, if, for, while	Misiones 0 y 2	S04_01_entorno_sintaxis
Funciones, assert, lambda	Misión 3	S04_02_funciones_modulos
Listas, diccionarios, comprensiones	Misión 1	S04_03_estructuras_datos
CSV, JSON, try/except	Misiones 4 y 5	S04_04_ficheros_excepciones
Clases y herencia (ampliación)	—	S04_05_poo_buenas_practicas

Los **apuntes** (1_Teoria/apuntes/) explican cada línea con ejemplos cuyo resultado puedes comprobar con una calculadora.

HOY HAS VISTO

Python, de cero a tu primer programa

Datos

Variables, listas y diccionarios

Lógica

if, **for** y funciones con **return**

Seguridad

assert y **try/except**

Ahora, a practicar: la misión 0 te espera.